FTP SERVER LINUX

Descargar PDF

- FTP SERVER LINUX
  - INTRODUCCIÓN
  - VSFTPD EN GNU/LINUX
    - CARACTERÍSTICAS PRINCIPALES
  - INSTALACIÓN Y PUESTA EN MARCHA
    - INSTALACIÓN
    - ARCHIVOS Y RUTAS HABITUALES
  - ESTRUCTURA DE VSFTPD.CONF
    - DIRECTIVAS MÁS IMPORTANTES
      - DIRECTIVAS GENERALES Y DE AUTENTICACIÓN
      - DIRECTIVAS DE ACCESO ANÓNIMO
      - USUARIOS LOCALES, VIRTUALES Y DIRECTORIOS
      - CONTROL DE USUARIOS Y COMANDOS
      - RED, PUERTOS Y TRANSFERENCIA
      - REGISTRO, LÍMITES Y RENDIMIENTO
      - TLS Y ENDURECIMIENTO
  - ESCENARIO 1: USUARIOS DEL SISTEMA
    - CREACIÓN DE USUARIOS FTP
    - CONFIGURACIÓN BÁSICA PARA USUARIOS LOCALES
    - LISTAS DE CONTROL DE ACCESO
    - CONFIGURACIÓN ESPECÍFICA POR USUARIO
  - ESCENARIO 2: ACCESO ANÓNIMO
    - CONFIGURACIÓN ANÓNIMA DE SOLO LECTURA
    - RESTRICCIÓN POR RED Y RENDIMIENTO
  - ESCENARIO 3: USUARIOS VIRTUALES
    - USUARIO REAL DE MAPEO
    - BASE DE CREDENCIALES CON PAM_USERDB
    - CONFIGURACIÓN PAM
    - INTEGRACIÓN EN VSFTPD
    - PREPARACIÓN DE DIRECTORIOS
    - CONFIGURACIONES POR USUARIO O POR PERFIL
  - CARPETAS COMPARTIDAS Y BIND MOUNT
    - APERTURA DE SESIÓN PAM
  - LIMITACIONES DE VSFTPD Y CÓMO COMPENSARLAS
    - GRUPOS FUNCIONALES Y PERFILES DE ACCESO
    - RESTRICCIONES POR HORARIO, ORIGEN O CONTEXTO
    - CUOTAS Y CONTROL DE ESPACIO
    - CRITERIO PRÁCTICO
  - MODO ACTIVO Y MODO PASIVO

- CONFIGURACIÓN DEL MODO PASIVO
- REGISTRO Y AUDITORÍA
- SEGURIDAD EN VSFTPD
  - MEDIDAS RECOMENDADAS
  - CIFRADO CON FTPS
  - SOBRE PAM Y LOS SCRIPTS DE SESIÓN
- COMPROBACIONES BÁSICAS
- CONCLUSIÓN
- REFERENCIAS

INTRODUCCIÓN

En sistemas GNU/Linux existen varios servidores FTP, pero el más habitual en entornos docentes y en muchas distribuciones generales es **vsftpd** (*Very Secure FTP Daemon*). Su popularidad se debe a tres ideas de diseño muy claras: **seguridad**, **rendimiento** y **simplicidad**. La propia documentación oficial lo presenta como un servidor pequeño, rápido y orientado a reducir la superficie de ataque.

Aunque FTP sigue siendo útil para estudiar el funcionamiento de los servicios de transferencia de archivos, conviene recordar una limitación importante: **FTP transmite credenciales y comandos en texto claro** si no se protege con TLS. Por ese motivo, en entornos reales suele preferirse **FTPS** cuando se quiere mantener compatibilidad con FTP, o **SFTP** cuando se trabaja sobre SSH.

Este documento se centra en la instalación y configuración de `vsftpd` en Linux, con especial atención a los aspectos de seguridad, autenticación y auditoría. Se presentan tres escenarios típicos: acceso de usuarios locales, acceso anónimo controlado y acceso mediante usuarios virtuales. Además, se incluyen recomendaciones para proteger el servicio y asegurar que las transferencias se registran adecuadamente.

VSFTPD EN GNU/LINUX

Entre las implementaciones más conocidas del servicio FTP en Linux destacan las siguientes:

- **vsftpd**. Prioriza la seguridad y el rendimiento. Ofrece soporte para usuarios locales, usuarios virtuales, restricciones por configuración, listas de acceso, límites de conexión y FTPS.
- **ProFTPD**. Es muy flexible y modular, pero su configuración suele ser más extensa y compleja.
- **Pure-FTPd**. También es una solución madura y segura, con buen soporte para usuarios virtuales y entornos compartidos.

En esta unidad trabajaremos con **vsftpd** porque cubre con solvencia los escenarios más habituales en un aula o en una pequeña infraestructura: acceso autenticado, acceso anónimo controlado, usuarios virtuales, enjaulado de usuarios, modo pasivo, auditoría y cifrado mediante TLS.

CARACTERÍSTICAS PRINCIPALES

- Funciona con **usuarios locales del sistema** y también con **usuarios virtuales** autenticados mediante PAM.
- Permite **enjaular usuarios** en su directorio de trabajo mediante `chroot`.
- Soporta **modo activo y modo pasivo**, algo esencial cuando hay firewalls o NAT.
- Puede registrar tanto **transferencias** como **comandos y respuestas del protocolo**.
- Admite **configuración por usuario** usando `user_config_dir`.
- Permite añadir **FTPS** para cifrar la sesión mediante TLS.

INSTALACIÓN Y PUESTA EN MARCHA

INSTALACIÓN

En distribuciones basadas en Debian, `vsftpd` se instala desde los repositorios oficiales con `apt`:

```
apt update
apt install vsftpd
```

Tras la instalación, el servicio queda registrado en `systemd` y normalmente se inicia automáticamente. Podemos comprobar su estado con:

```
systemctl status vsftpd
systemctl is-active vsftpd
```

ARCHIVOS Y RUTAS HABITUALES

Las rutas exactas pueden variar ligeramente entre distribuciones, pero en las distribuciones basadas en Debian es habitual trabajar con los siguientes elementos:

- `/etc/vsftpd.conf` - Archivo principal de configuración.
- `/etc/pam.d/vsftpd` - Servicio PAM usado por `vsftpd`.
- `/var/log/vsftpd.log` - Log propio de `vsftpd` si se usa el formato interno.
- `/var/log/xferlog` - Log de transferencias en formato estándar `xferlog`.
- `/srv/ftp` o `/var/ftp` - Ubicación habitual del contenido anónimo, según distribución o política local.
- `/var/run/vsftpd/empty` - Directorio vacío usado como `secure_chroot_dir`.

ESTRUCTURA DE VSFTPD.CONF

La configuración de `vsftpd` se basa en directivas con el formato:

```
opcion=valor
```

La sintaxis es estricta: **no debe haber espacios alrededor del signo `=`**. Los valores booleanos se expresan normalmente como `YES` o `NO`.

Configuración mínima orientativa

El siguiente ejemplo muestra una configuración básica en la que se permite el acceso de usuarios autenticados, se habilita la escritura y se enjaula a cada usuario en su directorio de trabajo.

```
local_enable=YES
write_enable=YES
chroot_local_user=YES
allow_writeable_chroot=YES
```

DIRECTIVAS MÁS IMPORTANTES

El fichero `vsftpd.conf` contiene muchas opciones, pero en la práctica la mayoría de configuraciones se apoyan siempre en un conjunto relativamente reducido. Para que la consulta sea más útil, se agrupan aquí por bloques funcionales.

DIRECTIVAS GENERALES Y DE AUTENTICACIÓN

Parámetro	Función	Observaciones
`anonymous_enable`	Habilita o deshabilita el acceso anónimo.	Si está en `YES`, se aceptan los usuarios `anonymous` y `ftp`.
`local_enable`	Permite el acceso de usuarios autenticados.	Necesaria para usuarios locales y también para usuarios virtuales.
`guest_enable`	Trata los usuarios autenticados como invitados mapeados.	Base del modelo de usuarios virtuales.
`guest_username`	Usuario real del sistema al que se mapean los usuarios virtuales.	Todas las sesiones virtuales se ejecutan con esa identidad.
`pam_service_name`	Nombre del servicio PAM que usará `vsftpd`.	Por defecto suele ser `vsftpd`.
`check_shell`	Comprueba si la shell del usuario es válida.	En compilaciones con PAM su efecto puede no ser relevante.
`session_support`	Abre y mantiene sesión PAM durante la conexión.	Útil si se usan módulos PAM de tipo `session`, por ejemplo `pam_exec`.

DIRECTIVAS DE ACCESO ANÓNIMO

Parámetro	Función	Observaciones
`anon_root`	Directorio raíz del acceso anónimo.	Debe tener permisos coherentes con el nivel de acceso deseado.
`write_enable`	Habilita o bloquea globalmente los comandos de escritura.	También afecta al usuario anónimo.
`anon_upload_enable`	Permite subir archivos de forma anónima.	Requiere además `write_enable=YES`.
`anon_mkdir_write_enable`	Permite crear directorios en acceso anónimo.	También requiere `write_enable=YES`.
`anon_other_write_enable`	Permite otras operaciones de escritura anónima, como borrar o renombrar.	Debe usarse con mucha cautela.
`anon_world_readable_only`	Limita las descargas anónimas a archivos legibles por todos.	Suele ser una protección razonable.
`no_anon_password`	Evita pedir contraseña al usuario anónimo.	Puede resultar cómodo en laboratorios, pero reduce trazabilidad.
`ftp_username`	Usuario del sistema asociado al acceso anónimo.	Por defecto suele ser `ftp`.

Parámetro	Función	Observaciones
`anon_max_rate`	Limita la velocidad de transferencia de usuarios anónimos.	El valor se expresa en bytes por segundo.
`anon_umask`	Máscara usada en archivos creados por el usuario anónimo.	Solo aplica si se permite escritura anónima.

USUARIOS LOCALES, VIRTUALES Y DIRECTORIOS

Parámetro	Función	Observaciones
`local_root`	Directorio inicial del usuario autenticado.	Puede ser fijo o usar `user_sub_token`.
`user_sub_token`	Sustituye un token como `$USER` por el nombre del usuario.	Muy útil con usuarios virtuales.
`user_config_dir`	Directorio de configuraciones específicas por usuario.	Contiene directivas de `vsftpd`, no scripts.
`chroot_local_user`	Enjaula usuarios locales o invitados en su raíz.	Medida habitual de aislamiento.
`allow_writeable_chroot`	Permite escritura en la raíz del `chroot`.	Cómodo, pero menos estricto desde el punto de vista de seguridad.
`chroot_list_enable`	Activa una lista de usuarios afectados por `chroot`.	Su comportamiento depende de `chroot_local_user`.
`chroot_list_file`	Ruta del fichero asociado a la lista `chroot`.	Por defecto hay una ruta predefinida.
`passwd_chroot_enable`	Deriva el `chroot` desde el directorio del usuario en `/etc/passwd`.	Útil en diseños muy concretos.
`virtual_use_local_privs`	Hace que los usuarios virtuales usen privilegios de usuarios locales.	Muy importante si los usuarios virtuales deben escribir.
`local_umask`	Máscara por defecto para usuarios autenticados.	`022` es un valor habitual en prácticas.

CONTROL DE USUARIOS Y COMANDOS

Parámetro	Función	Observaciones
`userlist_enable`	Activa una lista de control de usuarios.	Sirve para permitir o denegar según `userlist_deny`.
`userlist_file`	Ruta del fichero de lista de usuarios.	Puede usarse para limitar qué cuentas acceden por FTP.
`userlist_deny`	Define si la lista es de bloqueo o de permiso.	`NO` significa que solo se permiten los usuarios listados.

Parámetro	Función	Observaciones
`cmds_allowed`	Lista de comandos FTP permitidos.	Permite endurecer mucho el servicio.
`cmds_denied`	Lista de comandos FTP denegados.	Si un comando aparece en ambas listas, prevalece la denegación.
`download_enable`	Habilita o deshabilita las descargas.	Permite construir un servicio solo de subida.
`dirlist_enable`	Controla si se pueden listar directorios.	Puede utilizarse para ocultar la estructura remota.
`chmod_enable`	Permite usar `SITE CHMOD`.	Solo aplica a usuarios locales.
`dirmessage_enable`	Muestra mensajes al entrar en un directorio.	Busca por defecto un fichero `.message`.
`message_file`	Nombre del fichero de mensaje de directorio.	Solo tiene efecto si `dirmessage_enable=YES`.

RED, PUERTOS Y TRANSFERENCIA

Parámetro	Función	Observaciones
`listen`	Hace que `vsftpd` funcione en modo autónomo IPv4.	No debe combinarse con `listen_ipv6`.
`listen_ipv6`	Hace que `vsftpd` escuche en socket IPv6.	En muchos sistemas también acepta IPv4.
`listen_port`	Puerto TCP del servicio FTP.	Por defecto es el 21.
`pasv_enable`	Activa el modo pasivo.	Normalmente imprescindible tras NAT o firewall.
`pasv_min_port`	Puerto mínimo del rango pasivo.	Conviene abrirlo también en el firewall.
`pasv_max_port`	Puerto máximo del rango pasivo.	Debe formar un rango cerrado con el anterior.
`pasv_address`	Dirección anunciada por el servidor en modo pasivo.	Necesaria si el servidor está detrás de NAT.
`pasv_addr_resolve`	Permite resolver por DNS el valor de `pasv_address`.	Útil si se usa nombre en lugar de IP.
`port_enable`	Permite el modo activo.	En muchas redes modernas se usa menos que PASV.
`connect_from_port_20`	Hace que las conexiones de datos activas salgan desde el puerto 20.	Algunos clientes antiguos lo esperan.

Parámetro	Función	Observaciones
`idle_session_timeout`	Tiempo máximo de inactividad en la sesión de control.	Se expresa en segundos.
`data_connection_timeout`	Tiempo máximo permitido sin progreso en una transferencia.	También en segundos.
`ascii_upload_enable`	Permite subidas en modo ASCII.	En la práctica moderna suele dejarse desactivado.
`ascii_download_enable`	Permite descargas en modo ASCII.	Igual que el anterior, rara vez hace falta.

REGISTRO, LÍMITES Y RENDIMIENTO

Parámetro	Función	Observaciones
`xferlog_enable`	Registra transferencias.	Es la base del registro de actividad.
`xferlog_std_format`	Usa formato estándar `xferlog`.	Útil si se van a reutilizar herramientas externas de análisis.
`xferlog_file`	Ruta del log en formato `xferlog`.	Solo aplica con `xferlog_std_format=YES`.
`dual_log_enable`	Mantiene simultáneamente log estándar y log propio.	Muy útil en laboratorios.
`vsftpd_log_file`	Ruta del log propio de `vsftpd`.	Si `syslog_enable=YES`, puede dejar de usarse.
`log_ftp_protocol`	Registra comandos y respuestas FTP.	Muy útil para depuración.
`syslog_enable`	Envía el log al sistema `syslog`.	Alternativa al log en archivo propio.
`max_clients`	Limita el número total de clientes conectados.	Solo relevante en modo autónomo.
`max_per_ip`	Limita el número de clientes por dirección IP.	Útil para evitar abusos.
`max_login_fails`	Número máximo de fallos antes de cerrar la sesión.	Reduce intentos repetidos.
`local_max_rate`	Limita velocidad de usuarios autenticados.	Valor en bytes por segundo.
`trans_chunk_size`	Ajusta el tamaño de bloque en la transferencia.	Suele dejarse por defecto.

TLS Y ENDURECIMIENTO

Parámetro	Función	Observaciones
`ssl_enable`	Activa FTPS mediante SSL/TLS.	Requiere certificados y cliente compatible.

Parámetro	Función	Observaciones
`rsa_cert_file`	Ruta del certificado RSA del servidor.	En algunos sistemas certificado y clave pueden ir juntos.
`rsa_private_key_file`	Ruta de la clave privada RSA.	Debe protegerse con permisos correctos.
`allow_anon_ssl`	Permite FTPS para usuarios anónimos.	Solo aplica si `ssl_enable=YES`.
`force_local_logins_ssl`	Obliga a cifrar el login de usuarios autenticados.	Muy recomendable si se usa FTPS.
`force_local_data_ssl`	Obliga a cifrar el canal de datos de usuarios autenticados.	Complementa la protección del login.
`force_anon_logins_ssl`	Obliga a cifrar el login anónimo.	Poco habitual.
`force_anon_data_ssl`	Obliga a cifrar el canal de datos anónimo.	Poco habitual.
`require_ssl_reuse`	Exige reutilización de sesión SSL en el canal de datos.	Más seguro, pero puede romper algunos clientes.
`ssl_ciphers`	Restringe los cifrados permitidos.	Útil para endurecer la configuración.
`debug_ssl`	Registra información de depuración SSL.	Solo recomendable para diagnóstico.

No todas estas directivas serán necesarias en una misma instalación. Lo normal es combinar un subconjunto de ellas según el escenario: acceso anónimo controlado, usuarios locales, usuarios virtuales, integración con PAM o FTPS.

ESCENARIO 1: USUARIOS DEL SISTEMA

La forma más directa de trabajar con `vsftpd` es permitir el acceso a **usuarios locales del sistema**. Es un enfoque sencillo y suficiente para entornos pequeños o prácticas iniciales, ya que reutiliza la autenticación y los permisos del propio sistema operativo.

CREACIÓN DE USUARIOS FTP

Si queremos que una cuenta sirva solo para FTP y no permita abrir una shell interactiva, conviene asignarle como intérprete `/usr/sbin/nologin`.

Alta de un usuario FTP local

El siguiente ejemplo crea una cuenta pensada para uso exclusivo por FTP, con directorio propio y sin shell interactiva.

```
useradd -m -d /home/ftp/usuario1 -s /usr/sbin/nologin usuario1
passwd usuario1
```

En este ejemplo, el usuario `usuario1` tendrá como directorio personal `/home/ftp/usuario1`. A partir de aquí, el acceso FTP dependerá tanto de la configuración de `vsftpd` como de los permisos del sistema de archivos.

CONFIGURACIÓN BÁSICA PARA USUARIOS LOCALES

Configuración básica para usuarios locales

El siguiente ejemplo activa el acceso de usuarios locales, permite escritura y aplica un enjaulado sencillo sobre su directorio de trabajo.

```
local_enable=YES
write_enable=YES
local_umask=022
chroot_local_user=YES
allow_writeable_chroot=YES
```

Esta configuración activa el acceso autenticado, permite escritura y enjaula a cada usuario en su directorio raíz. Es una solución cómoda para prácticas, aunque desde un punto de vista más conservador resulta preferible que la raíz del `chroot` no sea escribible y que exista un subdirectorio específico para escritura.

Diseño más estricto del directorio

El siguiente ejemplo separa la raíz del entorno enjaulado del subdirectorio escribible para ajustar mejor la seguridad.

```
mkdir -p /home/ftp/usuario1/privada
chown root:root /home/ftp/usuario1
chmod 555 /home/ftp/usuario1
chown usuario1:usuario1 /home/ftp/usuario1/privada
chmod 755 /home/ftp/usuario1/privada
```

Con este diseño, el usuario queda enjaulado en `/home/ftp/usuario1`, pero escribe realmente en `/home/ftp/usuario1/privada`.

LISTAS DE CONTROL DE ACCESO

`vsftpd` puede permitir o denegar usuarios mediante listas. Es una medida útil cuando queremos reducir explícitamente qué cuentas pueden autenticarse por FTP.

Lista de usuarios permitidos

El siguiente ejemplo convierte el fichero de usuarios en una lista de permiso, de modo que solo podrán acceder las cuentas incluidas en él.

```
userlist_enable=YES
userlist_file=/etc/vsftpd.user_list
```

```
userlist_deny=NO
```

Con `userlist_deny=NO`, solo se permiten los usuarios que aparecen en el fichero indicado. Si se cambiara a `YES`, ocurriría lo contrario: los usuarios listados quedarían bloqueados.

CONFIGURACIÓN ESPECÍFICA POR USUARIO

La directiva `user_config_dir` permite crear **archivos de configuración por usuario**. Cada archivo contiene directivas de `vsftpd`, no scripts. Su utilidad principal es ajustar rutas, permisos o límites de forma individual.

Configuración individual

El siguiente ejemplo indica el directorio desde el que `vsftpd` leerá las configuraciones particulares de cada usuario.

```
user_config_dir=/etc/vsftpd/user_conf
```

Si existe el archivo `/etc/vsftpd/user_conf/usuario1`, sus directivas complementarán o sustituirán a las globales durante la sesión de `usuario1`.

ESCENARIO 2: ACCESO ANÓNIMO

El acceso anónimo permite conectarse con los usuarios `anonymous` o `ftp` sin usar una cuenta personal del sistema. Históricamente se ha utilizado para distribuir documentación o software de lectura pública.

Hoy en día debe habilitarse con mucha prudencia, porque reduce la trazabilidad y amplía la superficie de exposición del servicio.

CONFIGURACIÓN ANÓNIMA DE SOLO LECTURA

Acceso anónimo de solo lectura

El siguiente ejemplo habilita el acceso anónimo, fija un directorio público y bloquea cualquier operación de escritura.

```
anonymous_enable=YES
anon_root=/var/ftp/publica
write_enable=NO
anon_upload_enable=NO
anon_mkdir_write_enable=NO
anon_other_write_enable=NO
```

Esta es la configuración más razonable para un acceso anónimo: lectura sí, escritura no. Aquí conviene precisar el papel de `write_enable`: **no es una directiva exclusiva de usuarios autenticados**, sino un

interruptor general que permite o bloquea los comandos FTP que modifican el sistema de archivos. Por tanto, si `write_enable=NO`, tampoco habrá subidas, borrados o creación de directorios para usuarios anónimos.

Las directivas `anon_upload_enable`, `anon_mkdir_write_enable` y `anon_other_write_enable` actúan como permisos adicionales específicos del acceso anónimo, pero solo tienen efecto si antes se ha activado `write_enable=YES`. Dicho de otro modo: para que un usuario anónimo pueda escribir, deben cumplirse **las dos condiciones** siguientes:

- `write_enable=YES` para permitir operaciones de escritura a nivel global.
- La directiva anónima correspondiente en `YES`, junto con permisos adecuados en el sistema de archivos.

Por eso, en una configuración de solo lectura puede dejarse `write_enable=NO` sin problema. Si se necesitara permitir subidas anónimas, además de habilitar `anon_upload_enable=YES`, sería obligatorio activar `write_enable=YES` y ajustar cuidadosamente los permisos del directorio destino. En entornos docentes o productivos no suele ser recomendable salvo que exista una necesidad muy concreta.

RESTRICCIÓN POR RED Y RENDIMIENTO

En las prácticas de la unidad, el acceso anónimo se usa de forma restringida: solo desde una red concreta y con límite de velocidad. Ese planteamiento es razonable cuando el objetivo es exponer material público pero controlando el consumo de recursos.

Límite de velocidad para anónimo

El siguiente ejemplo impone un límite de transferencia a las sesiones anónimas para evitar consumos excesivos de ancho de banda.

```
anon_max_rate=625000
```

Ese valor limita la transferencia aproximada a 5 Mbit/s. Las restricciones por dirección IP suelen aplicarse con herramientas externas como `tcp_wrappers`, firewall o políticas de red.

ESCENARIO 3: USUARIOS VIRTUALES

Los **usuarios virtuales** son identidades que existen solo dentro del servicio FTP. No corresponden necesariamente a cuentas reales del sistema, lo que permite administrar un número grande de usuarios sin crear cuentas locales para cada uno.

En `vsftpd`, este modelo suele construirse combinando tres piezas:

- Una base de usuarios autenticados gestionada por **PAM**.
- Un **usuario real de mapeo** del sistema, que ejecuta las sesiones.
- Una estructura de directorios y permisos que aísla a cada usuario virtual.

USUARIO REAL DE MAPEO

El primer paso es crear una cuenta real sin shell interactiva. Esa cuenta no representa a un usuario humano; actúa como contenedor de las sesiones FTP virtuales.

Usuario de mapeo

El siguiente ejemplo crea el usuario real bajo el que se ejecutarán todas las sesiones de los usuarios virtuales.

```
useradd -r -m -d /home/ftp -s /usr/sbin/nologin ftpvirtual
```

BASE DE CREDENCIALES CON PAM_USERDB

Una opción clásica consiste en usar el módulo `pam_userdb`. Para ello se prepara un fichero de texto y se transforma a una base de datos tipo hash con `db_load`.

Es importante respetar el formato correcto del fichero: **usuario y contraseña en líneas alternas**, no en la misma línea.

Fichero de usuarios virtuales

El siguiente ejemplo muestra el formato correcto del fichero de credenciales antes de convertirlo en la base de datos usada por PAM.

```
pedro
pwd1
luis
pwd2
lucia
pwd3
```

```
db_load -T -t hash -f usuarios_virtuales.txt /etc/vsftpd/usuarios_virtuales.db
chmod 600 /etc/vsftpd/usuarios_virtuales.db
```

CONFIGURACIÓN PAM

Después se crea un servicio PAM específico, por ejemplo `/etc/pam.d/vsftpd_virtual`:

Servicio PAM para usuarios virtuales

El siguiente ejemplo define un servicio PAM específico que autentica y valida cuentas contra la base de datos generada previamente.

```
auth    required pam_userdb.so db=/etc/vsftpd/usuarios_virtuales
account required pam_userdb.so db=/etc/vsftpd/usuarios_virtuales
```

Obsérvese que en PAM se indica la ruta **sin** la extensión `.db`.

INTEGRACIÓN EN VSFTPD

Configuración de `vsftpd` para usuarios virtuales

El siguiente ejemplo integra PAM, el usuario de mapeo y el directorio dinámico de cada cuenta virtual dentro de la configuración principal.

```
local_enable=YES
write_enable=YES
guest_enable=YES
guest_username=ftpvirtual
virtual_use_local_privs=YES
pam_service_name=vsftpd_virtual
user_sub_token=$USER
local_root=/home/ftp/$USER
chroot_local_user=YES
allow_writeable_chroot=YES
```

Estas directivas hacen que:

- `local_enable=YES` permita autenticación no anónima.
- `guest_enable=YES` haga que los usuarios autenticados se traten como invitados mapeados.
- `guest_username=ftpvirtual` ejecute todas las sesiones bajo la cuenta real `ftpvirtual`.
- `virtual_use_local_privs=YES` otorgue a los usuarios virtuales privilegios equivalentes a los locales. Sin esta directiva, suelen comportarse con permisos de tipo anónimo, más restrictivos.
- `user_sub_token=$USER` y `local_root=/home/ftp/$USER` asignen automáticamente el directorio de cada usuario.

PREPARACIÓN DE DIRECTORIOS

Preparación de directorios base

El siguiente ejemplo crea la estructura inicial de trabajo de varios usuarios virtuales y la asigna al usuario de mapeo.

```
mkdir -p /home/ftp/pedro /home/ftp/luis /home/ftp/lucia
chown -R ftpvirtual:ftpvirtual /home/ftp
chmod 755 /home/ftp
```

Si se desea un diseño más seguro, puede repetirse la misma idea vista con usuarios locales: raíz no escribible y subdirectorio de trabajo dentro de ella.

CONFIGURACIONES POR USUARIO O POR PERFIL

`user_config_dir` resulta especialmente útil con usuarios virtuales, ya que permite aplicar ajustes diferentes a cada cuenta o reutilizar perfiles mediante enlaces simbólicos a archivos de configuración comunes.

Ajuste por usuario virtual

El siguiente ejemplo activa el directorio desde el que se aplicarán ajustes particulares para cada cuenta virtual.

```
user_config_dir=/etc/vsftpd/user_conf
```

Dentro de `/etc/vsftpd/user_conf/pedro` podríamos definir, por ejemplo, un `local_root` distinto o límites de ancho de banda específicos.

CARPETAS COMPARTIDAS Y BIND MOUNT

Cuando un usuario está enjaulado, los enlaces simbólicos hacia rutas externas no suelen resolver el problema de compartir contenido fuera del árbol del `chroot`. La técnica más sólida consiste en utilizar **montajes bind**.

Carpeta compartida dentro del espacio del usuario

El siguiente ejemplo monta una carpeta externa dentro del árbol visible por el usuario enjaulado mediante un `bind mount`.

```
mkdir -p /home/ftp/pedro/publica
mount --bind /var/ftp/usuarios/publica /home/ftp/pedro/publica
```

Si hay muchos usuarios, no conviene llenar `/etc/fstab` con decenas de montajes permanentes. En ese caso, una estrategia más flexible es abrir una **sesión PAM** y ejecutar scripts mediante `pam_exec` en el momento de inicio o cierre de sesión.

APERTURA DE SESIÓN PAM

Para que `vsftpd` invoque módulos de tipo sesión, debe habilitarse:

Apertura de sesión PAM en `vsftpd`

El siguiente ejemplo activa el soporte de sesión PAM para que puedan ejecutarse módulos de tipo `session` durante la conexión FTP.

```
session_support=YES
```

Y en el fichero PAM correspondiente podrían añadirse líneas como estas:

Ejecución de script con `pam_exec`

El siguiente ejemplo muestra cómo invocar un script del sistema al abrir la sesión mediante un módulo PAM de tipo `session`.

```
session optional pam_exec.so /usr/local/sbin/ftp-session-mount.sh
```

En este punto conviene distinguir bien dos mecanismos:

- `user_config_dir` sirve para **directivas de configuración de** `vsftpd`.
- `pam_exec` sirve para **ejecutar scripts del sistema** al abrir o cerrar la sesión.

LIMITACIONES DE VSFTPD Y CÓMO COMPENSARLAS

Frente a otros servidores FTP más grandes o más orientados a hosting compartido, `vsftpd` adopta una filosofía más austera. Esto tiene ventajas claras en simplicidad y seguridad, pero también implica que algunas funciones no aparecen como opciones nativas directas. En lugar de incorporar todas esas políticas dentro del propio demonio, muchas veces se delegan en **PAM**, en los **permisos del sistema de archivos** o en herramientas propias del sistema operativo.

GRUPOS FUNCIONALES Y PERFILES DE ACCESO

`vsftpd` no ofrece un sistema interno de grupos o perfiles equivalente al de otros servidores. Aun así, en la práctica se puede trabajar de forma muy parecida si se combinan varias técnicas:

- Configuraciones por usuario con `user_config_dir`, reutilizando perfiles comunes mediante enlaces simbólicos o archivos generados a partir de plantillas.
- Permisos Unix, grupos del sistema y ACL para decidir qué directorios puede leer o escribir cada usuario real o de mapeo.
- `bind mount` o `pam_mount` para presentar carpetas compartidas dentro del `chroot` solo a determinados usuarios o conjuntos de usuarios.

Con suficiente orden en la estructura de directorios, este enfoque permite simular bastante bien la idea de "grupo FTP" aunque `vsftpd` no la implemente como una entidad propia.

RESTRICCIONES POR HORARIO, ORIGEN O CONTEXTO

Otra diferencia habitual respecto a otros servidores FTP es que muchas restricciones de acceso avanzadas no se expresan en `vsftpd.conf`, sino en la pila PAM. Esto no es una debilidad en sí misma, pero desplaza la lógica de control a otro nivel de configuración.

En particular, los módulos PAM de tipo `account` permiten aplicar políticas que en otros productos aparecen integradas de serie:

- `pam_time` para limitar el acceso por franjas horarias.
- `pam_access` para permitir o denegar conexiones según origen, servicio o usuario.
- `pam_listfile` para reforzar listas blancas o negras de cuentas.
- `pam_mount` para preparar dinámicamente el entorno visible por el usuario durante la sesión.

Restricciones complementarias mediante PAM

El siguiente ejemplo muestra una pila PAM en la que se añaden controles de horario y de origen además de la autenticación habitual.

```
auth    required pam_listfile.so item=user sense=deny file=/etc/ftpusers onerr=succeed
account required pam_time.so
account required pam_access.so accessfile=/etc/security/access-ftp.conf
session optional pam_mount.so
```

La idea importante no es memorizar estos módulos, sino entender que `vsftpd` puede apoyarse en PAM para cubrir necesidades de control de acceso que el servidor no resuelve por sí solo. En entornos docentes y pequeños laboratorios esto ofrece mucha flexibilidad, pero exige documentar bien la solución para que siga siendo mantenible.

CUOTAS Y CONTROL DE ESPACIO

`vsftpd` tampoco incorpora un sistema propio de cuotas de disco por usuario o por grupo, algo que sí aparece de forma más visible en otros servidores FTP. En Linux, esa función suele resolverse en el nivel correcto: el **sistema de archivos**.

Por eso, si se quiere limitar el espacio consumido por usuarios FTP, lo habitual es recurrir a cuotas del propio sistema, por ejemplo con las herramientas clásicas de `quota`, `setquota` o `edquota`, o con cuotas de proyecto en sistemas de archivos que las soporten. De este modo, la restricción no depende solo del servicio FTP, sino de cualquier proceso que escriba en esa ruta.

Gestión externa de cuotas

El siguiente ejemplo ilustra una activación típica de cuotas a nivel de sistema, independiente de `vsftpd`, para limitar el espacio disponible a un usuario concreto.

```
apt install quota
edquota usuario1
quota -u usuario1
```

Este enfoque encaja bien con la filosofía de `vsftpd`: el servidor se ocupa de autenticar, aislar y registrar sesiones, mientras que el sistema operativo asume el control fino de recursos y políticas globales.

CRITERIO PRÁCTICO

En resumen, `vsftpd` no destaca por acumular funciones administrativas "todo en uno", sino por integrarse bien con los mecanismos estándar de GNU/Linux. Esa es precisamente la clave para solventar sus carencias aparentes: combinar el demonio FTP con **PAM**, **permisos y ACL**, **montajes bind o** `pam_mount` y **cuotas del sistema de archivos**.

MODO ACTIVO Y MODO PASIVO

FTP utiliza una conexión de control y otra de datos. Por eso la configuración de red es especialmente importante. En redes sencillas suelen funcionar ambos modos, pero en presencia de NAT o firewalls el **modo pasivo** suele ser el más fiable.

CONFIGURACIÓN DEL MODO PASIVO

Rango de puertos pasivos

El siguiente ejemplo habilita el modo pasivo y reserva un rango concreto de puertos de datos para facilitar la apertura en el firewall.

```
pasv_enable=YES
pasv_min_port=55000
pasv_max_port=56999
port_enable=YES
```

Con esta configuración, `vsftpd` seguirá aceptando modo activo y además reservará un rango concreto para conexiones pasivas. Ese rango debe abrirse también en el firewall.

Si el servidor está detrás de NAT, normalmente será necesario indicar la IP pública o el nombre resoluble que el servidor debe anunciar:

Dirección anunciada en modo pasivo

El siguiente ejemplo fija la dirección pública que el servidor anunciará a los clientes cuando trabaje en modo pasivo.

```
pasv_address=203.0.113.10
```

Si se utiliza un nombre DNS en lugar de una IP, habrá que acompañarlo de `pasv_addr_resolve=YES`.

REGISTRO Y AUDITORÍA

Una buena configuración de FTP no termina en la autenticación. También debe permitir saber **quién se conecta**, **qué transfiere** y **qué comandos ejecuta**. Para ello, `vsftpd` ofrece varios mecanismos de log.

Registro detallado de actividad

El siguiente ejemplo combina registro de transferencias, trazas del protocolo y un fichero de log propio para facilitar la auditoría.

```
xferlog_enable=YES
dual_log_enable=YES
log_ftp_protocol=YES
vsftpd_log_file=/var/log/vsftpd/ftp-server.log
```

Esta combinación resulta útil en entornos de laboratorio porque conserva información detallada de transferencias y de la conversación FTP. Si se prefiere integrar el registro con el sistema, puede utilizarse

`syslog_enable=YES`.

SEGURIDAD EN VSFTPD

MEDIDAS RECOMENDADAS

- Deshabilitar el acceso anónimo salvo que exista una necesidad real.
- Enjaular a los usuarios en su directorio de trabajo.
- Usar cuentas sin shell interactiva para accesos exclusivamente FTP.
- Limitar explícitamente los usuarios permitidos cuando el servicio no deba estar abierto a todas las cuentas del sistema.
- Restringir el rango de puertos pasivos y abrir solo los necesarios en el firewall.
- Revisar periódicamente los logs del servicio.

CIFRADO CON FTPS

Si se quiere evitar que las credenciales viajen en texto claro, debe activarse FTPS. Una configuración básica podría incluir lo siguiente:

Configuración básica de FTPS

El siguiente ejemplo activa TLS y obliga a cifrar tanto la autenticación como el canal de datos de los usuarios autenticados.

```
ssl_enable=YES
rsa_cert_file=/etc/ssl/certs/vsftpd.pem
rsa_private_key_file=/etc/ssl/private/vsftpd.key
force_local_logins_ssl=YES
force_local_data_ssl=YES
```

Con ello se obliga a cifrar tanto la autenticación como el canal de datos para usuarios autenticados. La documentación oficial advierte además de que algunos ajustes de TLS pueden afectar a la compatibilidad con determinados clientes, por lo que siempre conviene probar desde el cliente real que vaya a usarse.

SOBRE PAM Y LOS SCRIPTS DE SESIÓN

Cuando se integran scripts mediante PAM para montar carpetas, aplicar cuotas o registrar eventos, es importante que la pila PAM esté pensada para un servicio no interactivo como FTP. No todos los módulos de sesión que funcionan bien en un login de consola son apropiados para `vsftpd`.

COMPROBACIONES BÁSICAS

Una vez aplicada la configuración, conviene verificar siempre el servicio con un pequeño guion de comprobación:

```
systemctl restart vsftpd
systemctl is-active vsftpd
```

```
ss -ltnp | grep :21
tail -f /var/log/vsftpd/ftp-server.log
```

Después se deben probar, como mínimo, los siguientes casos:

- Inicio de sesión con un usuario autenticado.
- Subida y descarga de ficheros en su zona privada.
- Acceso a carpetas compartidas montadas dentro del `chroot`.
- Funcionamiento del modo pasivo desde un cliente situado tras otra red.
- Acceso anónimo, si está habilitado.
- Conexión FTPS, si se ha activado TLS.

CONCLUSIÓN

`vsftpd` es una solución muy adecuada para aprender y desplegar un servicio FTP en Linux con un nivel razonable de seguridad. Su configuración es compacta, pero suficientemente potente como para cubrir escenarios muy distintos: usuarios locales, acceso anónimo controlado, usuarios virtuales, directorios enjaulados, carpetas compartidas y sesiones cifradas.

La clave para administrarlo bien no está en memorizar directivas sueltas, sino en entender la relación entre cuatro elementos: **autenticación**, **permisos del sistema de archivos**, **topología de red** y **auditoría**. Cuando esos cuatro aspectos están alineados, el servicio resulta predecible, seguro y mucho más fácil de mantener.

REFERENCIAS

- Sitio oficial de `vsftpd`: https://security.appspot.com/vsftpd.html
- Manual oficial de configuración de `vsftpd.conf`: https://security.appspot.com/vsftpd/vsftpd_conf.html
- Página de manual de Debian para `vsftpd.conf`: https://manpages.debian.org/bookworm/vsftpd/vsftpd.conf.5.en.html